Projet ML : Prédiction du diabète

Comprendre et optimiser un modèle de classification (Random Forest)

GROUPE 2

Mayssa Moussa
Eya Makhlouf
Fida Sassi
Sarra Sghaier

Dataset : Pima Indian Diabetes · 768 observations · 8 variables médicales · 1 variable cible : diabète (0 / 1)

Introduction

- Ce projet présente l’implémentation et l’analyse d’un modèle de Random Forest appliqué au dataset Pima Indians Diabetes, un problème de classification binaire visant à prédire la présence ou l’absence de diabète.
- L’objectif est d’étudier le comportement du modèle à travers plusieurs étapes
- Enfin, nous comparons la Random Forest à des modèles classiques afin d’évaluer son efficacité et sa robustesse.

Présentation du dataset

Pima Indian Diabetes

Pima Indian Diabetes · Source : National Institute of Diabetes and Digestive and Kidney Diseases (NIDDK)

768
Observations

8
Features

2
Classes

Contexte Médical

Communauté : Femmes Pimas d'Arizona, USA

Critères : Âge ≥ 21 ans, sexe féminin uniquement

Source : NIDDK — Étude longitudinale sur le diabète

Collecte : Fin des années 1980 (données cliniques réelles)

Structure du fichier

Nb observations	768 lignes
Nombre de variables	8 variables descriptives 1 variable cible binaire (Outcome)
Variable cible	Outcome = 0 : non diabétique. Outcome = 1 : diabétique

Extrait du dataset

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1
5	5	116	74	0	0	25.6	0.201	30	0
6	3	78	50	32	88	31.0	0.248	26	1

PLan du cours

A1 Fondations

- Arbre de décision (CART)
- Bagging & Bootstrap
- Comparaison ID3 / C4.5 / CART

A2 Architecture

- Fonctionnement Random Forest
- Feature Randomness
- Dé-corrélation

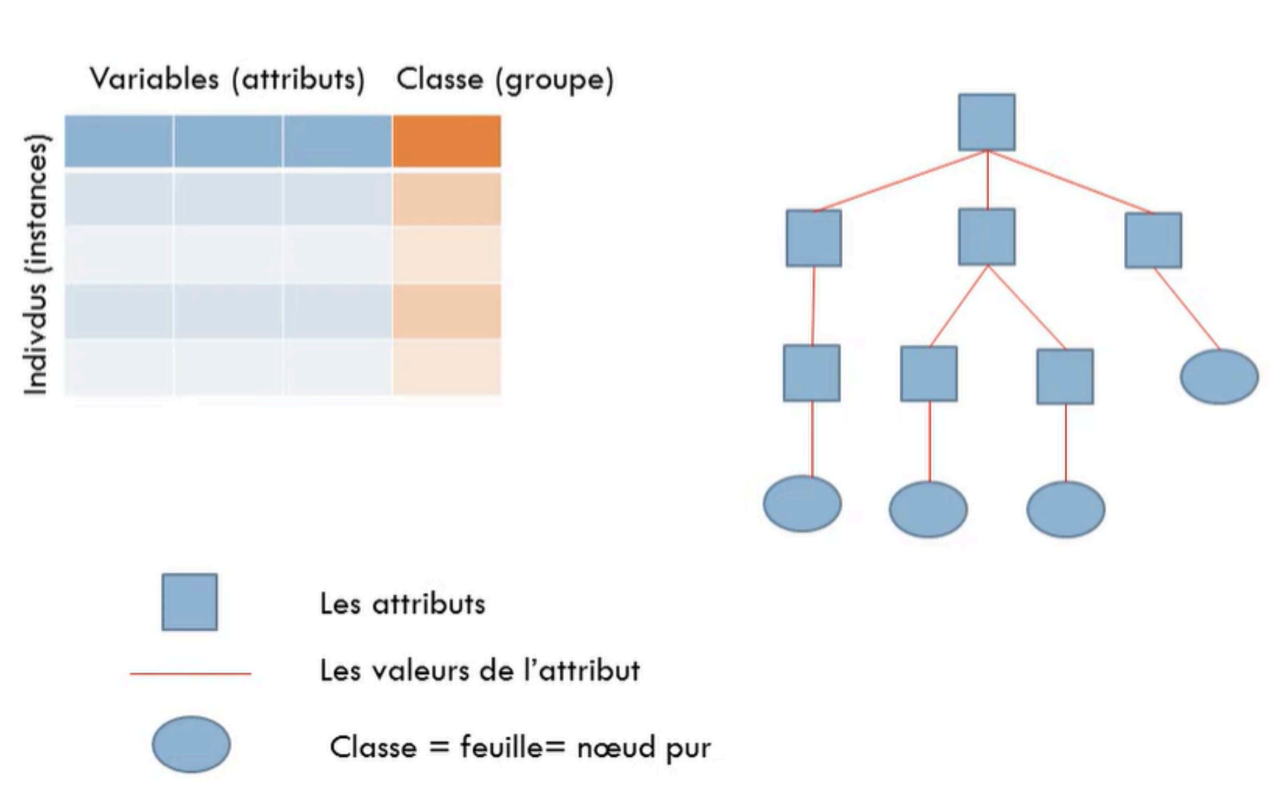
A3 Évaluation

- Erreur OOB (Out-Of-Bag)
- Feature Importances

A4 Optimisation

- n_estimators [10→500]
- max_features / max_depth
- convergence

Les arbres de décision : définition



Un arbre de décision est un modèle d'apprentissage supervisé pour **classifier** ou **prédire** des données à partir de leurs caractéristiques.

Structure de l'arbre (partie droite)

- **Nœud racine** :Point de départ de l'arbre, contient toutes les données.
- **Nœuds internes (carrés)** : Représentent des tests sur les attributs
- **Branches (lignes**) : Correspondent aux valeurs possibles d'un attribut (oui/non, petit/grand…).
- **Feuilles (ovales)** :Représentent la décision finale ou la classe.

A1 Fondations

Arbre de décision binaire · Bagging · Bootstrap

Arbre CART

- CART construit un arbre de manière récursive en posant des questions
- Les données sont divisés en deux parties uniquement
- Le processus continue jusqu'à obtenir des feuilles, et chaque feuille représente une classe finale

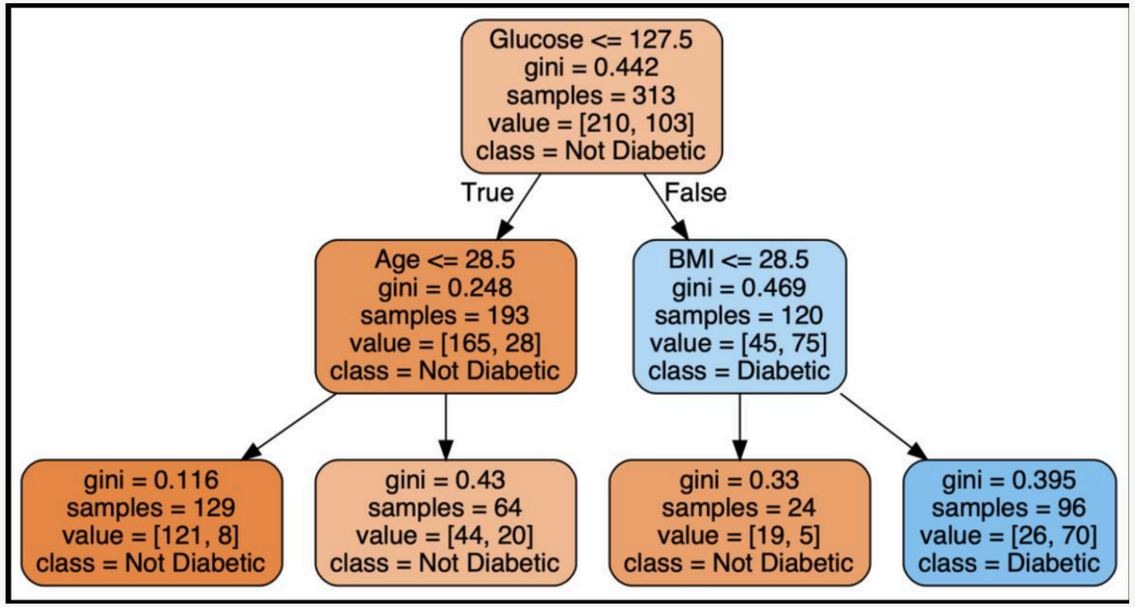
Bootstrap

- Le bootstrap consiste à tirer aléatoirement des échantillons à partir des données originales, avec remise.
- Certaines observations peuvent être répétées, d'autres absentes dans l'échantillon.
- chaque échantillon contient environ 63% des données uniques du dataset initial.

Bagging

- Bagging <=> Bootstrap Aggregating.
- L'idée est d'entraîner plusieurs arbres de décision indépendants, chacun sur un échantillon bootstrap différent.
- Ensuite, chaque arbre fait une prédiction, et la décision finale est obtenue par un vote majoritaire.

A1 —Principe d’un arbre de décision Binaire



Signification des éléments :

- gini : mesure d’impureté (plus petit = mieux)
- samples : nombre d’individus
- value : [non diabétique, diabétique]
- class : prédiction finale

Un arbre de décision binaire:

- Coupe les données en plusieurs sous-groupes
- Chaque nœud = une question (ex : Glucose <127.5 ?)
- Chaque feuille = une décision (0 ou 1)

Objectif : minimiser l’impureté des nœuds

- Pour choisir la meilleure question, on utilise l’indice de Gini.
- Plus le Gini est faible, plus le nœud contient une seule classe.
- L’algorithme essaye toutes les features et tous les seuils possibles, et sélectionne celui qui réduit le plus l’impureté moyenne.
- On répète récursivement jusqu’à un critère d’arrêt.
- Arrêt : profondeur max atteinte, ou nœud pur, ou trop peu d’échantillons

L’impureté - Lien avec Gini

L’impureté: mesure à quel point un groupe de données est mélangé entre les classes.

Exemple: Groupe 1 : que des “diabétiques” → pur
Groupe 2 : moitié diabétiques / moitié non → impur

lien avec Gini : Le critère de Gini mesure cette impureté :
Gini = 0 → groupe totalement pur
Gini élevé → groupe très mélangé

$$Gini = 1 - \sum p_k^2$$

Cas : 60% diabétiques, 40% non diabétiques

$$Gini = 1 - (0.6^2 + 0.4^2)$$
$$= 1 - (0.36 + 0.16)$$
$$= 1 - 0.52$$
$$= 0.48$$

A1 — Arbre de Décision Binaire : Comparaison ID3 / C4.5 / CART

CART→ Indice de Gini

- Mesure l'impureté : plus le Gini est proche de 0, plus le nœud est pur (une seule classe dominante).
- Maximum = 1−1/K quand toutes les classes sont équilibrées.

Choix du meilleur split

- Pour choisir le meilleur split, on calcule le **Gini moyen pondéré** des enfants.
- On minimise cette valeur.

Formule pour un nœud :

$$Gini = 1 - \sum_{k=1}^{K} p_k^2$$

- K = nombre de classes
- Pk= proportion d'exemples de la classe k dans le nœud

Gini moyen pondéré:

$$Gini_{split} = \frac{N_{gauche}}{N_{total}} \times Gini_{gauche} + \frac{N_{droite}}{N_{total}} \times Gini_{droite}$$

→ **0 = nœud pur , 0.5 = mélange équilibré**

A1 — Arbre de Décision Binaire : Comparaison ID3 / C4.5 / CART

ID3 = Iterative Dichotomiser 3

- **L'entropie** mesure le désordre (0 = parfaitement pur, log2(K)= maximum).
- **Le gain d'information** est la réduction d'entropie après le split. On choisit le split qui maximise ce gain.

Limite :

- favorise les attributs avec beaucoup de valeurs .

Entropie d'un nœud :

- K = nombre de classes
- Pk= proportion d'exemples de la classe k dans le nœud

$$Entropie = -\sum_{k=1}^{K} p_k \log_2(p_k)$$

$$\log_2(x) = \frac{\log(x)}{\log(2)}$$

Gain d'information pour un split :

$$Gain = Entropie_{parent} - \left(\frac{N_{gauche}}{N_{total}} Entropie_{gauche} + \frac{N_{droite}}{N_{total}} Entropie_{droite} \right)$$

A1 — Arbre de Décision Binaire : Comparaison ID3 / C4.5 / CART

C4.5→ Gain Ratio

- Corrige le biais du gain d'information (ID3) en divisant par l'entropie du split lui-même (SplitInfo).
- Pénalise les attributs avec trop de valeurs (car leur SplitInfo est élevé).
- On choisit le split avec le plus grand Gain Ratio (après avoir filtré ceux dont le gain est au moins égal à la moyenne).

Formule:

$$GainRatio = \frac{Gain}{SplitInfo}$$

Où :

$$SplitInfo = -\sum_{v \in valeurs} \frac{N_v}{N_{total}} \log_2 \left(\frac{N_v}{N_{total}} \right)$$

- v = chaque valeur possible de l'attribut (ou chaque branche).
- Nv= nombre d'exemples prenant la valeur.

A1 — Arbre de Décision Binaire : Comparaison ID3 / C4.5 / CART

Critère	ID3	C4.5	CART
Critère de split	Gain d'information	Gain Ratio	Gini (classification) / MSE
Type de variables descriptives	Catégorielles uniquement	Catégorielles + numériques	Tous types
Arbre binaire ?	Non (multi-branches)	Multi → souvent binaire	Toujours binaire
Élagage	Aucun	Post-élagage	Pré + Post élagage
Overfitting	Plus sensible	réduit	réduit
Valeurs numériques	ne gère pas directement les numériques (discrétisation obligatoire)	Gérées (probabilités)	très efficace sur numériques (splits binaires)
Type de problème	classification	classification	classification + régression (avec MSE pour régression)

Élagage (Pruning) dans les arbres de décision

L’élagage est une technique utilisée pour réduire la taille d’un arbre de décision en supprimant certaines branches.

Un arbre trop profond peut :

- apprendre tous les détails des données d’entraînement
- mais mal généraliser sur de nouvelles données

C’est ce qu’on appelle le surapprentissage (overfitting)

Comment ça marche ?
On supprime les branches qui apportent peu ou pas d’amélioration dans la prédiction.

Types d’élagage

1. Pré-élagage (Pre-pruning)
   - On arrête la croissance de l’arbre tôt
   - Exemple : profondeur max, nombre minimum d’échantillons

   Plus rapide, mais risque de sous-apprentissage

2. Post-élagage (Post-pruning)
   - On construit d’abord l’arbre complet
   - Puis on coupe les branches inutiles

   Plus précis et plus utilisé

A1 — Bagging et Bootstrap

Bootstrap (avec remise)

Échantillon Bootstrap **B1** (~63% uniques)

Échantillon Bootstrap **B2** (~63% uniques)

Échantillon Bootstrap **B3** (~63% uniques)

Arbre T1

Arbre T2

Arbre T3

Vote majoritaire

Prédiction finale
ŷ = classe majoritaire

Out-Of-Bag (OOB)

- ~37% des observations ne sont pas utilisées dans un bootstrap
- Ces données = échantillons OOB
- Elles servent de validation interne automatique

Avantage : OOB complète l'évaluation mais ne remplace pas totalement le test set

->oob_score=True → active le calcul OOB

OOB = approximation de la cross-validation, sans coût supplémentaire

A1 — Avantages et Limites de Bootstrap et Bagging

Les avantages de Bootstrap

- Permet de créer plusieurs échantillons à partir d'un seul dataset
- Utile quand les données sont limitées
- Simple et efficace

Les limites de Bootstrap

- Les échantillons peuvent être biaisés (doublons)
- Ne crée pas de nouvelles informations

Les avantages de Bagging

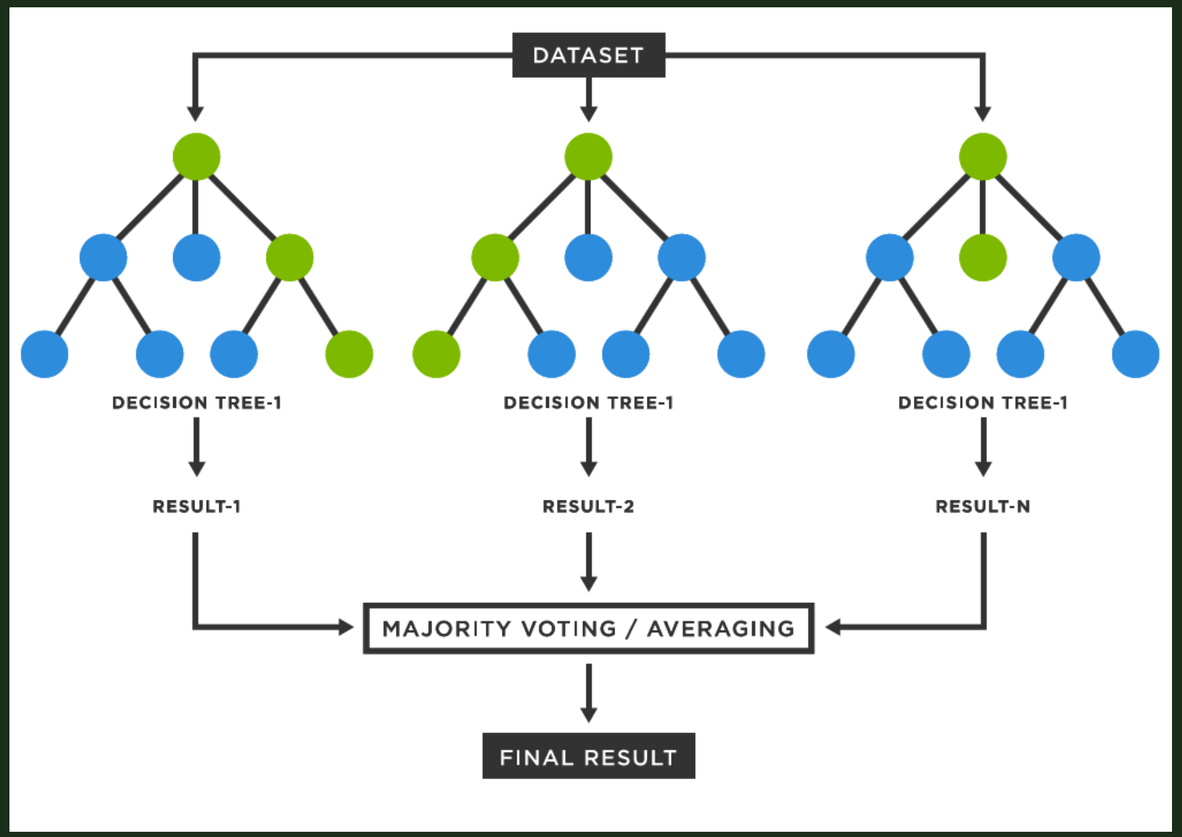
- Réduit la variance
- Améliore la stabilité du modèle
- Diminue le surapprentissage (overfitting)
- Améliore les performances globales

Les limites de Bagging

- Moins interprétable (plusieurs modèles)
- Coût de calcul élevé (plusieurs entraînements)
- Ne réduit pas le biais (juste la variance)

A2 Architecture

Random Forest · Schéma RF



Définition : Random Forest, une technique d'apprentissage automatique basée sur l'agrégation de plusieurs arbres de décision.

Objectif : Combiner plusieurs arbres pour améliorer la performance et la robustesse du modèle.

A2 — Feature Randomness · Dé-corrélation des arbres

1. Feature Randomness (randomisation des variables)
   - **Principe** : à chaque nœud d'un arbre, on ne considère qu'un sous-ensemble aléatoire de m variables (parmi p totales).
   - **Valeurs typiques** (classification) :
     - $m = \sqrt{p}$ (défaut sklearn) → ici $\sqrt{8} \approx 2$ ou 3
     - $m = \log_2(p)+1$ → ici=4
   - **Résultat** : décorrélation
     - Des arbres différents = des erreurs différentes.
     - La vote (moyenne) a une variance plus faible → meilleure généralisation du modéle.

A2 — Différence clé : Bootstrap vs Feature Randomness

Bootstrap

- Chaque arbre voit des données différentes
- Diversité via les données

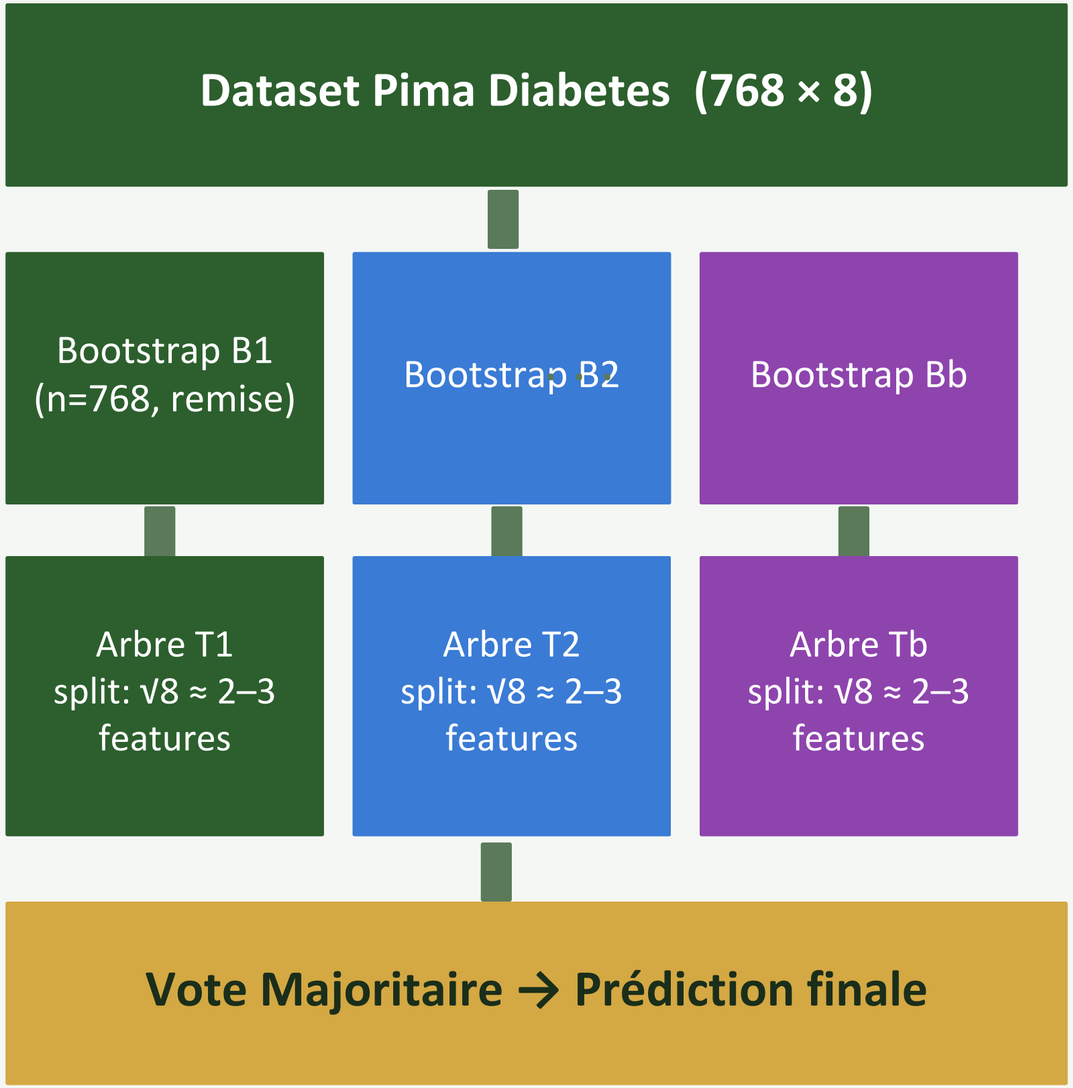
Feature Randomness

- chaque nœud voit des variables différentes
- Diversité via les variables

Résultat

Double diversité = arbres moins corrélés Meilleure robustesse du modèle

A2 — Fonctionnement d'une Forêt Aléatoire & Feature Randomness



Feature Randomness — Pima Dataset (p = 8 features)

- À chaque nœud : seul un sous-ensemble de m features est tiré aléatoirement.
- Le meilleur split est cherché uniquement sur ces m features.

- **Nombre total de features : p = 8**
- **Paramètre utilisé : max_features = 'sqrt'**
- **Donc : √8 ≈ 2 à 3 variables testées par split**

Impact : Moins de variables testées → arbres différents → réduction de la variance → meilleure performance

A3 Évaluation

Erreur OOB · Feature Importances · Résultats réels Pima Diabetes

OOB SCORE (Out-Of-Bag)

- L'OOB score (Out-Of-Bag score) est une méthode d'évaluation interne à la Random Forest.
- Évaluer la performance de la Random Forest sans séparer les données

Feature Importance

- Mesure l'importance de chaque variable dans la réduction de l'impureté (Gini)

Comparaison modèles

- Random Forest vs Decision Tree
- Permet de montrer le gain du bagging

Comprendre le Score OOB (Out-Of-Bag)

Une validation interne gratuite, propre au Random Forest — sans avoir besoin de cross-validation

Qu'est-ce que l'OOB ?

Lors du Bootstrap, chaque arbre est entraîné sur un tirage aléatoire avec remise d'environ 63% des données. Les 37% restants — ne sont pas utilisées par cet arbre — elles sont appelés échantillons Out-Of-Bag (OOB).

Formule & Interprétation

```
OOB Score  =  Proportion de prédictions OOB correctes
```

```
OOB Error  =  1 – OOB Score
```

→ OOB Score ≈ Accuracy sur le test set · Plus n_estimators ↑, plus l'OOB converge

FREE Gratuit & intégré

Pas besoin de séparer les données en train et test. Le score est calculé automatiquement pendant l'entraînement en activant oob_score=True.

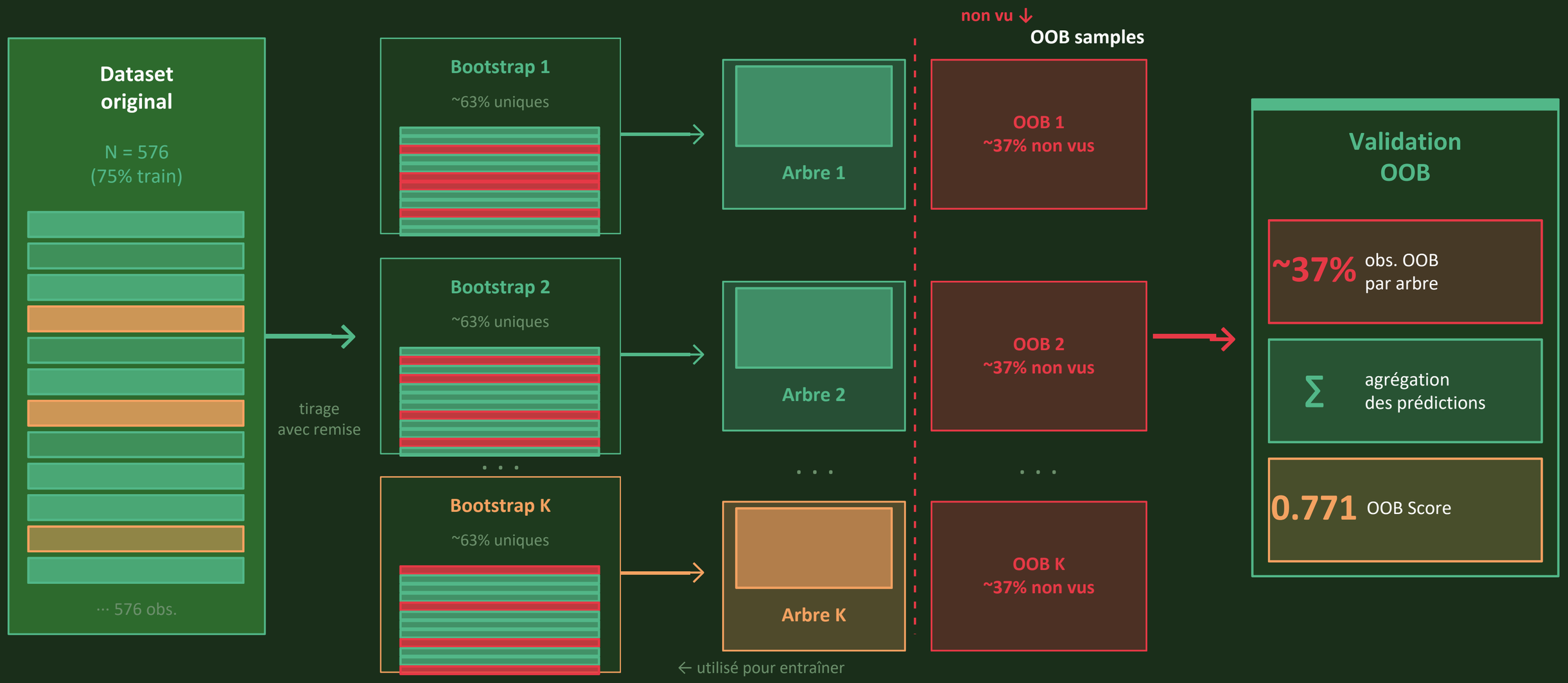
Mesure la convergence

En traçant l'OOB error en fonction du nombre d'arbres, on peut observer à partir de quel moment l'ajout d'arbres n'apporte plus d'amélioration. En général, la performance se stabilise autour de 100 à 200 arbres.

Estimateur non-biaisé

Chaque observation est prédite uniquement par des arbres qui ne l'ont jamais vue pendant l'entraînement→ équivalent à une leave-one-out cross-validation.

Schéma — Bootstrap & OOB : mécanisme complet



Idée clé : Chaque observation est évaluée uniquement par les arbres qui ne l'ont pas vue → validation non biaisée, proche d'une cross-validation.

A3 — Feature Importances

Quelles variables sont les plus importantes pour prédire le résultat ?

Principe

Le modèle regarde :

Chaque fois qu'une variable aide à mieux séparer les données dans un arbre

- si une variable fait une bonne séparation, elle est "récompensée"
- si elle est peu utilisée ou peu utile, elle a peu d'importance

①

Dans chaque arbre :

- on mesure combien une variable réduit le "désordre" (impureté Gini)
- plus la réduction est grande → plus la variable est importante

②

On répète ça :

- pour tous les arbres

③

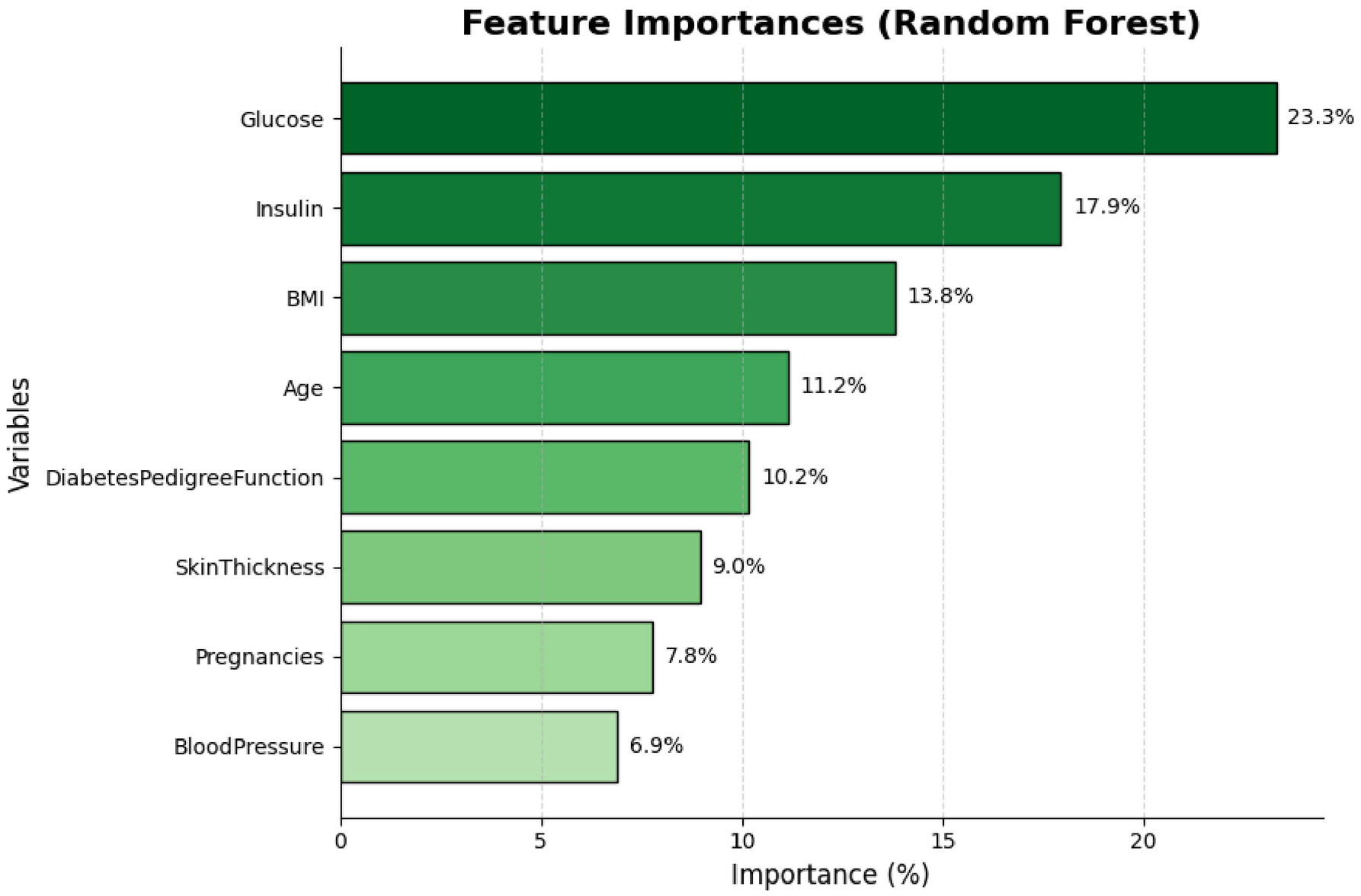
On additionne tout :

- puis on normalise pour avoir des valeurs entre 0 et 1

au final :

la somme des importances = 1

A3 — Feature Importances (Résultats réels Pima Dataset)



A3 — Random Forest vs Arbre de Décision

Arbre unique

- Forte variance
- Sensible au bruit
- Overfitting fréquent
- Dépend d'un seul modèle

Random Forest

- Agrégation de plusieurs arbres
- Bootstrap + feature randomness
- Réduction de la variance
- Modèle plus stable et robuste

Comparaison RF vs Arbre Unique :

Random Forest (n=100)	**0.7396**
Decision Tree	**0.6823**
Gain Bagging	**+5.7 pts**

Conclusion

Random Forest améliore les performances en réduisant la variance, donc meilleure généralisation.

A4 Optimisation

n_estimators · max_features · max_depth

n_estimators

- C’est le nombre d’arbres dans la forêt (Random Forest).
- [10→500]

max_depth

- C’est la profondeur maximale d’un arbre.
- [sqrt/log2],

max_features

- C’est le nombre de variables que le modèle regarde à chaque séparation dans un arbre.
- [None/3/5/10],

OOB_Score = true

A4 — Objectif de l'optimisation des hyperparamètres

Pourquoi optimiser le modèle ?

- Contrôler la complexité des arbres
- Favoriser la diversité des modèles
- Maximiser la capacité de généralisation

Paramètres étudiés

`n_estimators`	→ Nombre d'arbres dans la forêt
`max_depth`	→ Profondeur maximale d'un arbre
`max_features`	→ Sélection aléatoire des features
`oob_score`	→ Validation interne Out-Of-Bag

Biais vs Variance

Faible profondeur → underfitting (biais élevé)

Grande profondeur → overfitting (variance élevée)

Random Forest → réduit variance par agrégation

Objectif : trouver le meilleur compromis biais / variance

Méthodologie

- Analyse de l'OOB Score (validation interne)
- Analyse de l'Accuracy Test (généralisation)
- Étude de convergence du modèle

Idée clé : "Plus de complexité ≠ meilleur modèle" — Optimiser = trouver le juste équilibre

`n_estimators · max_depth · max_features · oob_score → les 4 leviers de la Random Forest`

A2 — Hyperparmètre : Nombre d’arbres (n_estimators)

Définition

- n_estimators = nombre d’arbres dans la forêt.
- Par défaut dans sklearn : 100.
- Chaque arbre est entraîné indépendamment → prédiction finale = vote majoritaire.

Effet sur la variance

- Plus d’arbres → réduction de la variance (stabilité).
- Mais au-delà d’un seuil (ex: 200–300 arbres), le gain est marginal.
- Pas de surapprentissage lié à n_estimators (contrairement à max_depth).

Compromis temps / performance

- n_estimators élevé → meilleure stabilité mais temps d’entraînement et de prédiction plus long.
- En pratique : choisir la valeur où la courbe se stabilise (ex: 200 pour Pima).

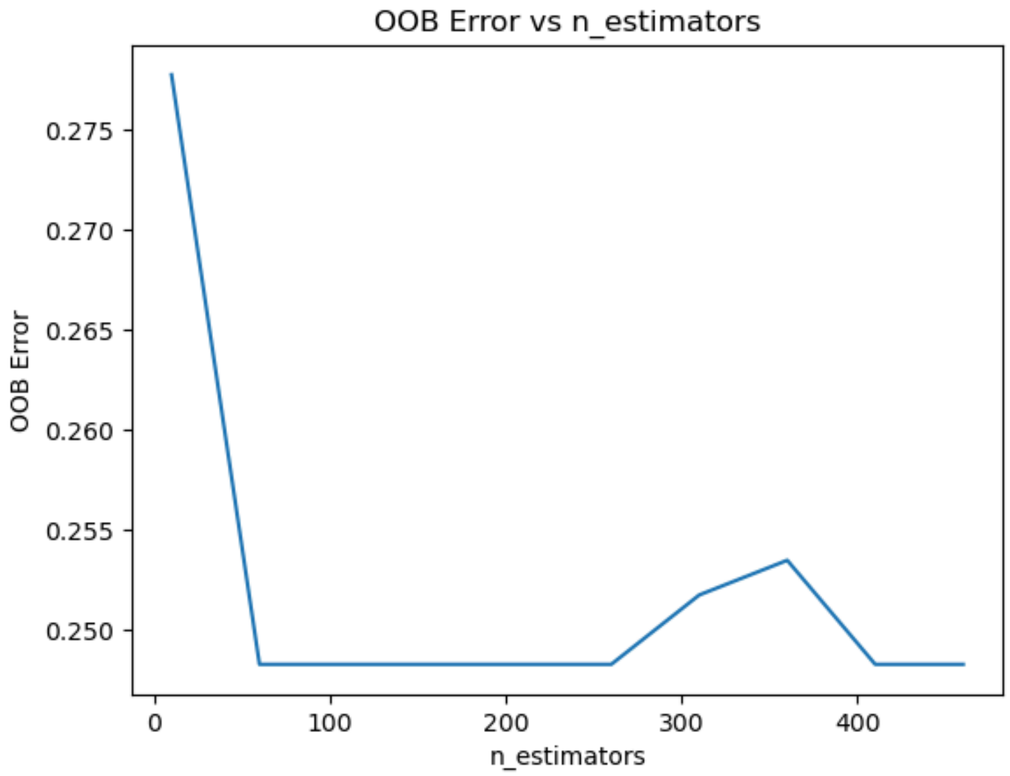
Idée clé
Augmenter le nombre d’arbres ne fait pas surapprendre,mais diminue la variance

A4 — Hyperparmètre : Nombre d’arbres (n_estimators)

n_estimators [10→500] — Convergence OOB Error

n_estimators	OOB Error	Observation
10	~0.285	Haute variance
50	~0.235	Amélioration rapide
100	**0.2431**	référence modèle
200	~0.240	gain marginal
500	~0.238	saturation

✓ *Convergence vers ~100 arbres*
✓ *OOB Score max ≈ 0.7569*
✓ *Gain marginal après 100*



A4 — Hyperparmètre : Profondeur de l’arbre (max_depth)

Définition

max_depth est un hyperparamètre qui définit la profondeur maximale de chaque arbre dans la Random Forest qui permet de :

- Contrôler la complexité des arbres
- Limiter le nombre de divisions (splits)
- Influencer directement le surapprentissage (overfitting)

Effet sur le modèle

Grande profondeur (ex : None)

- → arbres très complexes
- → risque d’overfitting

Petite profondeur (ex : 5, 10)

- → arbres plus simples
- → meilleure généralisation

Compromis Biais / Variance

Trop profond → modèle très précis mais instable
Trop petit → modèle trop simple (underfitting)

A4 — Hyperparmètre : Profondeur de l’arbre (max_depth)

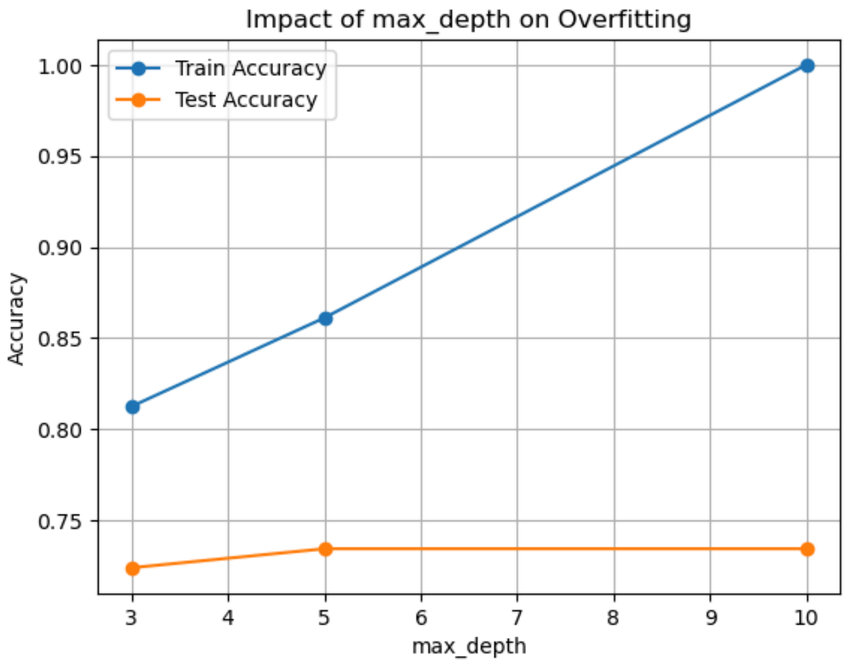
max_depth — Biais / Variance (Pima) :

max_depth	Train Acc	Test Acc	Interprétation
None	Train: ~1.000	Test: ~0.739	surapprentissage
5	Train: ~0.861	Test: ~0.734	Optimal
3	Train: ~0.812	Test: ~0.724	sous-apprentissage

Observation

- Train ↑ avec profondeur
- Test se stabilise → pas d’amélioration réelle

Conclusion : max_depth = 5 → meilleur compromis biais / variance



A4 — max_features (sqrt vs log2) - Impact sur la performance

max_features = nombre de variables testées à chaque split dans un arbre.

Résultats expérimentaux (Pima Dataset)

- *max_features = sqrt → Accuracy = 0.7396*
- *max_features = log2 → Accuracy = 0.7240*

On remarque que sqrt donne de meilleures performances que log2 sur ce dataset.

- max_features contrôle la diversité des arbres
- sqrt introduit un bon équilibre entre :
  - diversité des splits
  - stabilité des modèles
- log2 réduit trop le nombre de variables testées → perte d'information

Conclusion : **sqrt** est le meilleur choix pour ce dataset et correspond aussi au choix par défaut de sklearn

A2 — Hyperparmètre : Poids des classes (class_weight)

Définition

C’est un hyperparamètre qui permet d’attribuer un poids différent à chaque classe lors de l’apprentissage afin de corriger le déséquilibre des donnée

Objectif

- Améliorer les performances sur les classes minoritaires
- Éviter qu’un modèle favorise uniquement la classe majoritaire
- Rendre la classification plus équilibrée

Exemple

Dataset déséquilibré :

- Classe 0 : 90%
- Classe 1 : 10%

=>

Avec class_weight :

- Classe 0 → poids faible
- Classe 1 → poids élevé

Valeurs possibles :

- **None** → poids égaux(par défaut)
- **balanced** → poids automatiques selon les classes
- **dictionnaire** → poids manuels

Exercice Pratique

Reproduire un pipeline complet de Random Forest pour la classification du diabète et analyser ses performances.

1 Prétraitement

Charger, nettoyer,
KNN Imputer

2 Modélisation

RandomForest
oob_score=True

3 Évaluation

OOB, Feature
Importances, Courbes

Exercice — Consignes : Reproduire le Pipeline Complet

Étapes à réaliser (dans l'ordre)

1 Chargement & exploration

Charger le CSV Pima, df.info(), df.describe()
Identifier les 0 aberrants (Glucose, BMI, BP, Skin, Insulin)

2 Prétraitement (pipeline exact du cours)

Split 75/25 stratifié → Remplacer 0 par NaN
Médiane (Glucose/BP/BMI) + KNN Imputer k=5 (Skin/Insulin)
StandardScaler : fit_transform train, transform test

3 RF de base + comparaison

RandomForestClassifier(n_estimators=100, oob_score=True)
Comparer avec DecisionTreeClassifier
→ Afficher OOB Score et classification_report

4 Feature Importances (graphique)

rf.feature_importances_ → barh chart trié
Quelle variable est la plus importante ?

5 Courbe OOB Error vs n_estimators

Tester n ∈ range(10, 501, 10)
Tracer la courbe → identifier le point de convergence

Bonus :
Tester max_depth=[3,5,10]
Tracer Train vs Test Acc
Identifier le surapprentissage

Résultats attendus

OOB Score RF	~0.757
Accuracy RF (test)	~0.740
Accuracy Dec.Tree	~0.682
Feature #1	Glucose
Feature #2	Insulin
Convergence OOB	n ≈ 80-100